DSA-ASSESSMENT

GROUP 10

KIC- HNDCSAI-261-F-001,004,014,28

PART A-CONCEPTUAL UNDERSTANDING

1.What is a Stack and a Queue?

Stack:A Stack us a LIFO(Last in First Out) data structure.The last element you PUT in is the FIRST one to come out.You can only access the TOP element.You cannot reach anything below it without removing what is on top first.

There are three core operations:

push(x) — Add element x to the top of the stack.
pop() — Remove AND return the top element.
peek() — Look at the top element WITHOUT removing it

Queue:A Queue is a FIFO(First in First Out) data structure.The FIRST element you add is the FIRST one to come out.You can only ADD to one end and REMOVE from the other end.

There are four core operations:

enqueue(x) — Add element x to the REAR (back) of the queue.
dequeue() — Remove AND return the element from the FRONT.
peek() — Look at the FRONT element WITHOUT removing it.
isEmpty() — Is the queue empty? (front == -1)
isFull() — Is the array-based queue at capacity? (rear == length-1)

2.Give three real world applications for each.

Stack:

1.You can add a plate on TOP.You can always take from the TOP.You cannot take the bottom plate without removing all plates above it.

2.Each page you visit is pushed onto a stack.Clicking Back removes the most recent page off.You can only go back one step at a time.

3.When you press Ctrl+z, it removes the last action.The most recent thing is undone first.

Queue:

1.In a supermarket, first one to join the line is the first to be served.New people join at the back.

2.In a Government Hospital,The first patient to register is the first patient to be assigned to beds.

3.In the OS,the OS lines up processes and executes the one that is been waiting longest first.

PART B

1.ROBOT MOVEMENT UNDO SYSTEM

Scenario Overview

Based on the Scenario,The robot navigates in a grid.Its starting position is at the centre cell(column 3,row 3,0-indexed).Every move command-UP,DOWN,LEFT and RIGHT shifts the robot one cell in the chosen direction and pushes the command string onto a stack. The Undo operation pops the most recent command and applies the inverse direction, restoring the robot to its previous cell. The grid is re-displayed after every move and every undo.

How it works?

1. The robot starts at the center of a 7×7 grid at position (3,3), and the stack is empty.
2. The robot moves UP to position (3,2), and "UP" is pushed onto the stack → [UP].
3. The robot moves UP again to position (3,1), and another "UP" is pushed → [UP, UP].
4. The robot moves RIGHT to position (4,1), and "RIGHT" is pushed → [UP, UP, RIGHT].

5. The robot moves RIGHT again to position (5,1), and "RIGHT" is pushed → [UP, UP, RIGHT, RIGHT].
6. The robot moves DOWN to position (5,2), and "DOWN" is pushed → [UP, UP, RIGHT, RIGHT, DOWN].
7. Undo is performed: "DOWN" is popped from the stack, and its opposite (UP) is applied, moving the robot back to (5,1) → [UP, UP, RIGHT, RIGHT].
8. Undo is performed: "RIGHT" is popped, and its opposite (LEFT) is applied, moving the robot to (4,1) → [UP, UP, RIGHT].
9. Undo is performed: "RIGHT" is popped, and its opposite (LEFT) is applied, moving the robot to (3,1) → [UP, UP].

Algorithm Design

Step	Action
1	Robot starts at (x=3, y=3) on a 7×7 grid. Stack is empty.
2	move(dir) is called: compute new (nx, ny); validate bounds; update (x,y); push dir.
3	displayGrid() renders the 7×7 grid with 'R' at current position.
4	undo() is called: guard isEmpty(); pop last direction; apply inverse; displayGrid().
5	If the stack is empty and undo() is called → print 'Nothing to undo' and return.

2.ONLINE GAME MATCHMAKING

Scenario Overview

Players waiting for an online game session are managed usinga custom array-backed Queue.Each player carries three attributes:Username,rank and level. Players join the queue via enqueue() (FIFO order). When enough players are available, the system calls dequeue() to pull the longest-waiting player out and form a game session. A skill-based priority variant would extend this with a priority queue.

How it works?

1. The matchmaking system starts with an empty queue, where front = -1 and rear = -1, indicating that no players are waiting.
2. Player **Abdullah (Gold, 32)** joins the queue using enqueue; both front and rear are set to 0 →
   [Abdullah]
3. Player **John (Silver, 15)** joins the queue; rear moves to 1 →
   [Abdullah, John]
4. Player **Sara (Platinum, 48)** joins the queue; rear moves to 2 →
   [Abdullah, John, Sara]
5. A match is formed using dequeue; **Abdullah** (the first player) is removed from the front →
   Queue becomes **[John, Sara]**

6. After dequeue, front moves from 0 to 1, making **John** the new front (next player to be matched).
7. If another match is formed, **John** is dequeued and removed from the queue →
   Queue becomes **[Sara]**
8. Now front and rear both point to **Sara**, indicating only one player remains in the queue.
9. If the final player (**Sara**) is dequeued, the queue becomes empty again, and both front and rear are reset to -1 →
   [ ]

Algorithm Design

Step	Action
1	MatchmakingQueue created with capacity 5. front = –1, rear = –1 (empty).
2	enqueue(player) is called: if isFull() print warning and return; if isEmpty() set front = 0; assign player to queue[++rear].
3	displayQueue() iterates from front to rear, printing each player's details.
4	dequeue() is called: guard isEmpty(); save queue[front]; if front == rear reset both to –1 (queue now empty); else front++; return saved player.
5	If queue is empty and dequeue() is called → print 'Queue Empty' and return null.

Conclusion

This project demonstrates how both **Stack** and **Queue** data structures solve real-world problems efficiently by using their unique processing orders.

The **Stack (LIFO)** was used in the robot movement undo system, where the most recent action is reversed first. Each movement is pushed onto the stack, and the undo operation pops the last command and applies its opposite, allowing the robot to return to its previous position. This makes stack ideal for features like undo/redo, where actions must be reversed in the exact opposite order they were performed .

In contrast, the **Queue (FIFO)** was used in the online game matchmaking system, where players are processed in the order they arrive. Each player is enqueued when they join, and dequeued when a match is formed, ensuring that the longest-waiting player is served first. This guarantees fairness and order, which is essential in systems like matchmaking, scheduling, and service queues .